OpenCPO Architecture Remediation & QA Resolution Guide

Comprehensive Technical Specification for Resolving Session Isolation, Client-Side Injection, TCP Socket Exhaustion, and Real-Time Dashboard Fleet Stability.

AUTHOR / ROLE
Staff QA & Reliability Eng

CLASSIFICATION
Engineering Resolution (ADR)

TARGET VERSION
OpenCPO Admin v1.4.2

VERIFICATION STATUS
Verified & Actionable

1. Executive Summary

The OpenCPO real-time operations dashboard delivers mission-critical supervision across EV charging networks. An earlier implementation exhibited **continuous buffering, UI reloading traps, and frozen loading skeletons** caused by uncoordinated multi-component polling, un-deduplicated backend API queries (25 calls every 3 seconds), and destructive full-DOM replacements.

While initial stabilization decoupled the polling cadence and introduced in-place reconciliation, a comprehensive QA penetration and concurrency audit uncovered four critical failure modes requiring immediate architectural remediation before production deployment:

SEC-01	SEC-02	NET-01	CON-01
SESSION IDENTITY BLEED	CLIENT-SIDE DOM XSS	SSE SOCKET STARVATION	SYNC BACKOFF LOCKOUT

2. Risk Summary Table

ID	VULNERABILITY / BUG	SEVERITY	ROOT CAUSE	RESOLUTION APPROACH
SEC-01	Cross-Session Identity Bleed	`HIGH / P1`	Global 2.5s TTL cache stores request-bound `user` object.	Re-bind `request.state.user` per-request from cloned context.
SEC-02	DOM XSS via Charger Metadata	`HIGH / P1`	Dynamic row creation embeds unescaped string literals into `innerHTML`.	Implement RFC-compliant HTML entity sanitization on all client strings.
NET-01	HTTP/1.1 Socket Starvation	`MEDIUM / P2`	Multi-tab persistent SSE streams consume browser's 6-connection pool.	Enforce HTTP/2 multiplexing on ingress; add Page Visibility pause/resume.
CON-01	Manual Sync Backoff Lockout	`MEDIUM / P2`	`refreshNow()` fails to reset `consecutiveErrors` counter.	Reset error counter and in-flight flags on explicit operator refresh.

3. Deep-Dive Remediation Blueprint

SEC-01: Cross-Session Identity Bleed via Shared Cache

CVSS: 6.8 (MEDIUM) AFFECTED FILE: OPENCPO-ADMIN/ROUTES/DASHBOARD.PY

Detailed Problem Analysis:
To eliminate database thrashing, `_get_dashboard_context()` was equipped with a 2.5-second in-memory cache keyed by `(days, is_simulated, region_filter, selected_tz)`. However, the cached dictionary included the caller's `request.state.user` payload. When User A (Admin) queries the dashboard, User A's identity is cached. If User B (Operator) queries within the 2.5-second window with matching parameters, User B receives User A's identity object in the template context.

> **Security Impact:** Identity spoofing in the top navigation, cross-tenant metadata exposure, and transient elevation of UI capability badges.

The Best Approach:
Keep cached data strictly to shared telemetry (chargers, grid capacity, energy metrics), and dynamically stamp the active `request.state.user` onto a shallow copy of the context dictionary at dispatch time.

Exact Code Implementation:

```
# File: opencpo-admin/routes/dashboard.py

async def _get_dashboard_context(request: Request) → dict:
    """Fetch complete live data with safe per-request user re-binding."""
    days = int(request.query_params.get("days", 7))
    sim_param = request.query_params.get("simulated")
    is_simulated = (sim_param == "1" or sim_param.lower() == "true") if sim_param is not None else None
    region_filter = request.query_params.get("region", "all")
    selected_tz = request.query_params.get("tz", "IST")

    cache_key = (days, is_simulated, region_filter, selected_tz)
    now = time.time()

    # 1. Check cache hit
    cached = _dashboard_cache.get(cache_key)
    if cached and (now - cached["ts"]) < _DASHBOARD_CACHE_TTL:
        # CRITICAL FIX: Clone context and attach current request's user
        ctx = dict(cached["ctx"])
        ctx["user"] = getattr(request.state, "user", {}) or {}
        return ctx

    # 2. In-flight request deduplication
    if cache_key in _dashboard_inflight:
        shared_ctx = await _dashboard_inflight[cache_key]
        ctx = dict(shared_ctx)
        ctx["user"] = getattr(request.state, "user", {}) or {}
        return ctx

    loop = asyncio.get_event_loop()
    future = loop.create_future()
    _dashboard_inflight[cache_key] = future

    try:
        shared_ctx = await _build_dashboard_context(request, days, is_simulated, region_filter, selecte
        _dashboard_cache[cache_key] = {"ctx": shared_ctx, "ts": time.time()}
        if not future.done():
            future.set_result(shared_ctx)

        ctx = dict(shared_ctx)
```

```
        ctx["user"] = getattr(request.state, "user", {}) or {}
        return ctx
    except Exception as exc:
        if not future.done():
            future.set_exception(exc)
        raise
    finally:
        dashboard_inflight.pop(cache_key, None)
```

SEC-02: Client-Side DOM XSS in `createFleetRowElement`

CVSS: 7.2 (HIGH) AFFECTED FILE: OPENCPO-ADMIN/TEMPLATES/DASHBOARD.HTML

Detailed Problem Analysis:
When newly connected chargers appear on the network, `FleetLiveManager` creates a new row via `createFleetRowElement(cp)`. The method interpolated raw properties (`${cp.site}`, `${cp.vendor}`, `${cp.model}`) directly into an HTML string literal. An attacker registering a virtual or physical charge point with payload characters (e.g., `<img src=x onerror=...>`) achieves stored script execution on administrator browsers.

The Best Approach:
Introduce a lightweight, zero-dependency HTML entity encoder in the frontend and sanitize all string properties before string template interpolation.

Exact Code Implementation:

```
// File: opencpo-admin/templates/dashboard.html

function escapeHtml(str) {
    if (str === null || str === undefined) return '';
    return String(str)
        .replace(/&/g, '&')
        .replace(//g, '>')
        .replace(/"/g, '"')
        .replace(/'/g, ''');
}

function createFleetRowElement(cp) {
    const tr = document.createElement('tr');
    tr.className = 'fleet-row';
    tr.id = `fleet-row-${escapeHtml(cp.id)}`;
    tr.dataset.status = escapeHtml(cp.status);
    tr.dataset.id = escapeHtml(cp.id);
    tr.dataset.model = `${escapeHtml(cp.vendor || 'ABB')} ${escapeHtml(cp.model || 'Terra 54')}`;
    tr.dataset.site = escapeHtml(cp.site || 'Central Depot');

    const isOnline = cp.status === 'online';
    const safeId = escapeHtml(cp.id);
    const safeSite = escapeHtml(cp.site || 'Central Depot');
    const safeVendor = escapeHtml(cp.vendor || 'ABB');
    const safeModel = escapeHtml(cp.model || 'Terra 54');
    const safeVersion = escapeHtml(cp.ocpp_version || '1.6');
    const safeHeartbeat = escapeHtml(cp.last_heartbeat || 'Just now');

    // Connector Pills with strict sanitization
    let connHtml = '<div class="flex items-center gap-1.5 flex-wrap">';
    let hasLive = false;
    if (cp.live) {
        for (const [k, v] of Object.entries(cp.live)) {
            if (k.startsWith('connector_') && k.endsWith('_status')) {
                hasLive = true;
                const cid = escapeHtml(k.replace('connector_', '').replace('_status', ''));
                const statusStr = escapeHtml(v);
                let badgeClass = 'bg-slate-100 text-slate-600';
                if (v === 'Charging') badgeClass = 'bg-blue-50 text-blue-700 border border-blue-200 ani
                else if (v === 'Available') badgeClass = 'bg-emerald-50 text-emerald-700 border border-
                else if (v === 'Faulted') badgeClass = 'bg-red-50 text-red-700 border border-red-200';

                connHtml += `<span class="px-1.5 py-0.5 rounded text-[9px] font-mono font-bold ${badgeC
            }
        }
```

```
    }
    if (!hasLive) {
        connHtml += '<span class="px-1.5 py-0.5 rounded text-[9px] font-mono bg-emerald-50 text-emerald
    }
    connHtml += '</div>';

    tr.innerHTML = `
        <td class="font-mono font-bold text-blue-600 text-xs">
            <a href="/remote/${safeId}" class="hover:underline flex items-center gap-1 text-blue-600 no
                <i data-lucide="zap" class="w-3.5 h-3.5 inline-block"></i>
                <span>${safeId}</span>
            </a>
        </td>
        <td class="font-mono text-[11px] text-slate-500">${safeVersion}</td>
        <td class="fleet-status-cell">
            <span class="inline-flex items-center gap-1 px-2 py-0.5 rounded text-[10px] font-semibold $
                <span class="w-1.5 h-1.5 rounded-full ${isOnline ? 'bg-emerald-400 animate-pulse' : 'bg
                ${escapeHtml(cp.status.toUpperCase())}
            </span>
        </td>
        <td>
            <div class="text-slate-800 font-medium text-xs">${safeSite}</div>
            <div class="text-[10px] text-slate-500">${safeVendor} ${safeModel}</div>
        </td>
        <td class="fleet-connectors-cell">${connHtml}</td>
        <td class="font-mono text-[10px] text-slate-500 fleet-heartbeat-cell">${safeHeartbeat}</td>
        <td class="text-right fleet-actions-cell">
            <div class="inline-flex items-center gap-1 justify-end">
                <button hx-post="/action/start/${safeId}?connector=1" hx-target="#toast-container" hx-s
                    <i data-lucide="play" class="w-3 h-3 fill-current inline-block btn-icon-default"></
                    <span class="btn-spinner"></span>
                    <span>Start</span>
                </button>
                <button hx-post="/action/stop/${safeId}?connector=1" hx-target="#toast-container" hx-sw
                    <i data-lucide="square" class="w-3 h-3 fill-current inline-block btn-icon-default">
                    <span class="btn-spinner"></span>
                    <span>Stop</span>
                </button>
                <button hx-post="/action/reset/${safeId}?type=Soft" hx-target="#toast-container" hx-swa
                    <i data-lucide="rotate-ccw" class="w-3 h-3 inline-block"></i>
                    <span>Reset</span>
                </button>
                <button hx-post="/action/unlock/${safeId}?connector=1" hx-target="#toast-container" hx-
                    <i data-lucide="unlock" class="w-3 h-3 inline-block"></i>
                    <span>Unlock</span>
                </button>
            </div>
        </td>
    `;
    return tr;
}
```

NET-01: HTTP/1.1 Socket Starvation from Multi-Tab SSE

`CVSS: 5.3 (MEDIUM)` `AFFECTED AREA: OPENCPO-ADMIN/TEMPLATES/BASE.HTML & INGRESS PROXY`

Detailed Problem Analysis:
Standard browser connection pools enforce a ceiling of 6 TCP sockets per host under HTTP/1.1. In operations centers where staff maintain 3–4 dashboard tabs across monitoring displays, each tab holds a permanent socket for `EventSource('/api/events/stream')`. When background pollers (HTMX and `FleetLiveManager`) issue requests, they enter a stalled state awaiting socket liberation, mimicking an application buffer freeze.

The Best Approach:

1. **Ingress Protocol Enforcement:** Terminate SSL at the reverse proxy (Traefik / Nginx / Caddy) with HTTP/2 (`h2`) enabled. HTTP/2 multiplexes hundreds of SSE streams and concurrent fetch requests across a single TCP connection.
2. **Page Visibility Lifecycle Management:** In `base.html`, pause SSE streaming and polling timers when a browser tab is placed in the background (hidden), reconnecting instantly upon tab focus.

Visibility Hook Implementation (base.html):

```
// File: opencpo-admin/templates/base.html

document.addEventListener('visibilitychange', function() {
    if (document.hidden) {
        // Tab inactive: close SSE stream to conserve client socket pool
        if (window._sseInstance) {
            window._sseInstance.close();
            window._sseInstance = null;
        }
    } else {
        // Tab visible: reconnect instantly
        if (typeof window.initGlobalLiveStream === 'function') {
            window.initGlobalLiveStream();
        }
        if (window.FleetLiveManager) {
            window.FleetLiveManager.refreshNow();
        }
    }
});
```

CON-01: Manual Sync Backoff Lockout

`CVSS: 4.0 (MEDIUM)` `AFFECTED FILE: OPENCPO-ADMIN/TEMPLATES/DASHBOARD.HTML`

Detailed Problem Analysis:
When network lag occurs, `FleetLiveManager` scales its polling interval from 5s to 30s using exponential backoff. If an operator clicks the manual **Sync** button, `refreshNow()` executes an immediate fetch but previously failed to reset `consecutiveErrors = 0`. If that single refresh experienced transient latency, subsequent polls remained throttled at 30 seconds.

The Best Approach:
Explicit user action must reset the backoff penalty immediately and cancel any pending stale timers.

Exact Code Implementation:

```
// File: opencpo-admin/templates/dashboard.html

return {
    start: function() {
        scheduleNext();
    },
    refreshNow: function(qs) {
        if (qs) activeQs = qs;
        consecutiveErrors = 0; // Immediate backoff reset on user action
        clearTimeout(pollTimer);
        fetchLiveFleet();
    },
    destroy: function() {
        clearTimeout(pollTimer);
        if (abortCtrl) abortCtrl.abort();
    }
};
```

4. QA Verification & Test Suite Matrix

TEST CASE ID	TEST SCENARIO	EXECUTION STEPS	PASS / FAIL CRITERIA
TC-AUTH-01	Concurrent Identity Isolation	Simulate 50 parallel requests from Admin and Viewer sessions within 500ms.	**PASS:** Context reflects correct `request.state.user` for every individual session. Zero leakage.
TC-XSS-01	OCPP Payload Sanitization	Inject `<script>alert(1)</script>` into charger model/site payload via simulated boot.	**PASS:** Rendered in DOM as encoded entities; zero script execution in console.
TC-NET-01	Multi-Tab Socket Saturation	Open 6 dashboard tabs simultaneously; inspect Chrome DevTools Network timing.	**PASS:** Under HTTP/2, all tabs stream and poll concurrently with 0ms socket queuing.
TC-UX-01	Search Box Focus Persistence	Type text in `#fleet-search` while background poll completes.	**PASS:** Input cursor remains active, search query is never wiped, zero layout shift.

5. Engineering Rollout Checklist

- [x] **Backend Deduplication:** In-flight locks and 2.5s TTL cache deployed in `routes/dashboard.py`.
- [x] **DOM Stability:** Full-DOM innerHTML swapping replaced with in-place cell reconciliation.
- [x] **Session Protection:** Context shallow-cloning applied to ensure per-request identity isolation.
- [x] **Input Sanitization:** Entity encoding integrated across all client-rendered charger rows.
- [x] **Cadence Recovery:** Manual refresh resets backoff penalties immediately.